A restricted Narya prototype of modal K

1. Scope

Narya can express a two-mode modal-K interface resembling part of the boxing discipline in *Towards Type Systems That Reinterpret Themselves*. It does **not** thereby implement that paper's family $\Box_\kappa$: the native modal datatype carries computational values, not preserved object-language syntax. This note isolates the useful fragment and the remaining implementation obligations. It is not a formalisation or soundness proof of the paper's full calculus.

The comparison uses the paper at revision 5bdb198[1] and Narya at revision 0612a6f.[2] The paper's simply typed system has closed-value reinterpretation but no box modality. Here we consider the dependent system's $\Box_\kappa$ and distinguish it from the *computational* global-elements modality discussed as an alternative.

2. The modal-K interface

Narya's built-in `-functor` mode theory has two modes and one generating modality. Renaming that modality to `q` gives $q : \mathsf{DomType} \to \mathsf{CodType}$. These are distinct modes, **not** successive universe levels. The annotation `:q|` checks an argument behind a modal lock. The following definitions were accepted by Narya:

```
def Box (A :q| DomType) : CodType :=
  data [ box. (_ :q| A) ]

def K (A B :q| DomType)
  (f : Box (A -> B)) (x : Box A) : Box B :=
  match f, x [
  | box. g, box. a |-> box. (g a) ]
```

The constructor has the modal introduction interface `(a :q| A) -> Box A`, not an unrestricted `A -> Box A`. Matching exposes `g` and `a` with modal annotations; the argument of `box.` supplies the lock under which their application is legal. For closed domain-mode types A, B, this gives the familiar instance $\mathsf{Box}(A \to B) \to \mathsf{Box}(A) \to \mathsf{Box}(B)$. Closed domain-mode terms can be introduced using this constructor.

The one-way arrow cannot compose with itself. In particular, `Box (Box A)` is ill-moded: `Box A` belongs to `CodType`, whereas the argument of `Box` must belong to `DomType` behind the lock. This reproduces a useful no-nested-box restriction, but is not a proof that all of Narya's modal contexts coincide with the paper's single-lock contexts. The interface follows Narya's modal rules[3] and its upstream functor example.[4]

3. Computation is the decisive difference

Narya also accepts these two reflexivity witnesses:

Date: October 21, 2026.

[1] https://github.com/TurtlePU/imtt/tree/5bdb198c86a0b0a6f7407985068e2cd5d29c9bd1/latex

[2] https://github.com/gwaithimirdain/narya/tree/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f

[3] https://github.com/gwaithimirdain/narya/blob/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f/docs/source/modal.rst

[4] https://github.com/gwaithimirdain/narya/blob/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f/test/black/modal.t/functor.ny

```
def K_beta (A B :q| DomType)
  (f :q| A -> B) (a :q| A)
  : Id (Box B)
      (K A B (box. f) (box. a)) (box. (f a)) :=
  refl (box. (f a))

def beta_in_box (A :q| DomType) (a :q| A)
  : Id (Box A)
      (box. (((x : A) |-> x) a)) (box. a) :=
  refl (box. a)
```

For a concrete closed type, both commands below print `box. star. : Box One`:

```
def One : DomType := data [ star. ]

echo K One One (box. (x |-> x)) (box. star.)
echo (box. (((x : One) |-> x) star.) : Box One)
```

The `beta_in_box` definition exhibits an important *limitation*, not evidence of intensional quotation: a native beta-redex inside `box.` is definitionally equal to its result. Narya's normaliser[5] evaluates modal constructor arguments in a locked environment. Locks control access to variables; they do not suspend computation.

By contrast, the paper deliberately forbids computation inside quoted terms and boxed types, while allowing splices to compute. Its interpreter needs the unreduced construction in order to interpret it in a chosen model. A modal wrapper around a native value cannot recover distinctions already erased by native conversion. The prototype is therefore closer to the discussion's computational modal interface; it supplies neither the paper's intensional quotation nor arbitrary model reinterpretation.

4. What a faithful implementation would still need

- **Level-indexed syntax.** The bound κ covers all universe levels used in a construction, including its redexes, not only its result type. Current Narya has a self-containing universe at each mode (for example, `DomType : DomType`), with universe levels still planned; modes cannot stand in for this bound. See the universe documentation.[6]
- **The paper's typing and splicing rules.** Box formation checks the enclosed type under a lock. A faithful embedding must represent contexts, permitted outer splices and the absence of nested quotation and reinterpretation in quoted code, rather than accepting arbitrary ambient native types.
- **Interpretation in nominal models.** The paper interprets dependent syntax in internal categories with families. Conversion requires transports along equality witnesses extracted using a fixed reduction strategy. Neither `Box` nor `K` provides this interpreter or those witnesses. Replacing propositional model equations with native definitional equations changes the intended class of models.

An explicit object-language implementation remains plausible: define syntax, universe tags and typing derivations as data, then implement splicing and interpretation by recursion. Narya supports mutual and inductive-inductive definitions,[7] useful ingredients for such a project. The paper's program-specific translation into functions of finitely many models suggests another

[5] https://github.com/gwaithimirdain/narya/blob/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f/lib/core/norm.ml#L382-L400

[6] https://github.com/gwaithimirdain/narya/blob/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f/docs/source/built-in-types.rst

[7] https://github.com/gwaithimirdain/narya/blob/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f/docs/source/mutual-definitions.rst

research route. Neither route has been implemented here, and neither identifies arbitrary native Narya terms with reinterpretable syntax.

5. Verification status

Concatenating the three code blocks above into `prototype.ny` and running `narya -no-reformat -functor -modalities q prototype.ny` with the pinned revision succeeds (exit 0). All definitions are accepted without holes or axioms; the two `echo` commands return the result stated above. The general reflexivity witnesses check the displayed endpoints by conversion, including native beta-reduction inside a box.

Three separate negative checks, each appended to the accepted definitions, exit 1 as expected:

- `def capture (A :q| DomType) (x : Box A) : Box A := box. (match x [ box. a |-> a ])` is rejected with E1705: the ordinary variable `x` cannot be used behind the `q` lock without a key.
- `def unbox (A :q| DomType) (x : Box A) : A := match x [ box. a |-> a ]` is rejected with E1705: a `q`-annotated variable cannot be used without the corresponding lock/key.
- `def nested (A :q| DomType) : CodType := Box (Box A)` is rejected with E1700: `CodType` and `DomType` do not match.

These are rejections of particular programs, not general nondefinability theorems. The upstream functor example also checks successfully. The runs used OCaml 5.3.0 on macOS arm64, on 21 October 2026.

These checks establish only the prototype's acceptance and computation in that implementation. Narya currently documents self-containing universes and missing termination/positivity checks[8]; these results are not a consistency certificate.

The PDF was built with LuaLaTeX (TeX Live 2026), using the separately supplied MLTT-S Markdown-LaTeX runtime.[9] No packages, styles or build infrastructure accompany this note.

[8] https://github.com/gwaithimirdain/narya/blob/0612a6f935eb35fef6ff0bbf3afae8e6ff83c70f/docs/source/mutual-definitions.rst

[9] https://github.com/akuklev/MLTT-S/tree/ce465be